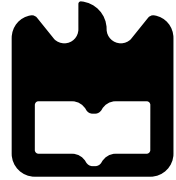


universidade de aveiro



deti

departamento de eletrónica,
telecomunicações e informática

Distributed Systems

Coordination

Eurico Pedrosa <efp@ua.pt>

2nd semester - 2025'26

Coordination in Distributed Systems

Coordination in Distributed Systems

The primary objective of **coordination** is to allow processes to cooperate and synchronize, ensuring they do not simultaneously access shared resources and can agree on the relative ordering of events

- **Relation to Synchronization:** Coordination and synchronization are closely related
 - **Process synchronization** ensures one process waits for another, and data synchronization ensures consistency between data sets
 - **Coordination** acts as an encapsulation of these synchronization tasks
- **The Coordination Perspective:** From an architectural standpoint, coordination is what happens “under the hood” to allow applications to execute. It describes the fundamental tasks processes must carry out to act as a single, coherent system

Challenges and Primary Coordination Tasks

Coordination is significantly more complex in distributed environments than in uniprocessor systems due to the **lack of a global clock and shared memory**.

- **Clock Synchronization:**

- Processes must coordinate to compensate for the absence of a global clock
- **Physical Clocks:** Synchronizing to actual real-world time (e.g., UTC)
- **Logical Clocks:** Ensuring agreement on the relative ordering of events
 - e.g., Lamport's "happens-before" relation

Challenges and Primary Coordination Tasks

- **Distributed Mutual Exclusion:** Ensuring that multiple processes cooperate to grant temporary exclusive access to shared resources, such as files or hardware
- **Election Algorithms:** The process by which a group of distributed processes appoints one specific process to act as a coordinator or leader
- **Gossip-based Coordination:** Utilizing decentralized, probabilistic information dissemination for tasks like aggregation and overlay construction

Clock Synchronization

Clock Synchronization

In a centralized system, time is unambiguous because all processes query a single operating system clock, ensuring that events are recorded in a consistent sequence.

- **The Distributed Challenge:** Distributed systems lack a global clock; instead, each machine maintains its own local timer, making agreement on the current time non-trivial
- **Clock Skew:** Even if nodes are initially synchronized, local timers gradually drift apart due to differences in hardware frequency, resulting in inconsistent time values across the system
- **Core Objective:** The goal of clock synchronization is to ensure that the deviation between the respective clocks of any two machines remains within a specified, acceptable bound

Clock Synchronization: Motivation

Without synchronization, a distributed system may assign a later event a timestamp that is earlier than a preceding event.

- **The Unix `make` Utility Case:**

- The `make` program uses file modification timestamps to decide which source files require recompilation
- If a source file is edited on a machine with a slow clock, its timestamp might be earlier than the existing object file, causing the utility to erroneously skip the compilation

Clock Synchronization: Motivation

- **Systemwide Requirements:**
 - **Financial Brokerage:** Accurate timestamps are required to determine the precise order of trades and transactions
 - **Security Auditing:** Reliable timing is essential for maintaining a valid chronological log of system events and accesses
 - **Performance Measurement:** Proper synchronization allows for the comparison of performance metrics collected across different physical nodes
- **Coordination and Consistency:** Many distributed coordination tasks rely on an accurate account of time to ensure that processes act as a single, coherent system

Physical Clocks

Despite being called “**clocks**,” computer time-keeping devices are technically timers based on precisely machined quartz crystals.

- **Hardware Mechanism:**

- ▶ Quartz crystals oscillate at a stable frequency when kept under tension
- ▶ A timer circuit includes two registers: a **counter** and a **holding register**
- ▶ Each oscillation decrements the counter; when it reaches zero, a CPU interrupt (clock tick) is generated, and the counter is reloaded from the holding register

Physical Clocks

- **Software Integration:**
 - At every clock tick, the interrupt service procedure increments a counter in memory
 - This software clock tracks the number of ticks since a known starting date
 - e.g., the Unix epoch
- **Clock Skew:** In a distributed system, individual crystal oscillators run at slightly different rates due to hardware variations, leading to diverging time values across machines

Physical Clocks: Global Time

- **International Atomic Time (TAI):**
 - Computed by averaging the ticks of Cesium 133 atomic clocks across multiple labs
 - Stable but diverges from solar time because the Earth's rotation is slowing down
- **Coordinated Universal Time (UTC)** (the civil timekeeping):
 - Based on atomic seconds but synchronized with the Earth's rotation
 - **Leap Seconds:** Introduced whenever the discrepancy between TAI and solar time reaches 800 milliseconds to keep noon aligned with the sun's highest point
- **UTC Dissemination:**
 - Distributed via shortwave radio stations (± 10 msec accuracy) and satellites like GPS
 - Ground-based time servers using satellite receivers can achieve accuracies within 50 ns

Clock Synchronization Algorithms

The goal is to maintain consistency between local software clocks across a distributed system. With UTC time t , the value of the software clock on machine p is denote by $C_{p(t)}$.

- **Precision (π)** (internal synchronization):

- The maximum deviation between the clocks of any 2 machines within the system

$$\forall t, \forall p, q : |C_{p(t)} - C_{q(t)}| \leq \pi$$

- **Accuracy (α)** (external synchronization):

- The maximum deviation between a machine's clock and an external reference like UTC

$$\forall t, \forall p : |C_{p(t)} - t| \leq \alpha$$

Clock Synchronization Algorithms

In a perfect world, we would have $C_{p(t)} = t$ for all p and all t , and thus $\pi = \alpha = 0$.

Unfortunately, hardware clocks, and thus also software clocks, are subject to **clock drift**.

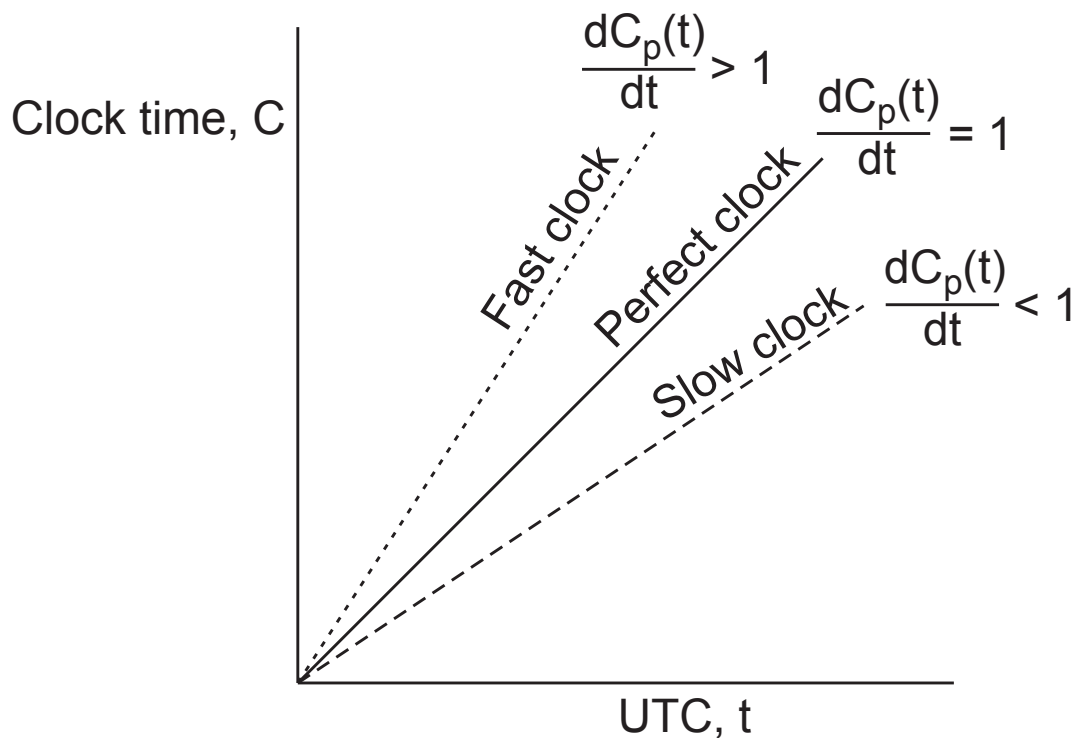
- **Clock Drift:** Hardware oscillators drift due to environmental factors (temperature, etc.)
 - A typical drift rate ρ is 10^{-6} seconds per second
- **Drift Condition:** The relative error of the actual frequency compared to the ideal frequency must stay within the bound ρ . Mathematically, this is expressed as:

$$1 - \rho \leq \frac{F(t)}{F} = \frac{\partial C_p(t)}{\partial t} \leq 1 + \rho$$

Clock Synchronization Algorithms

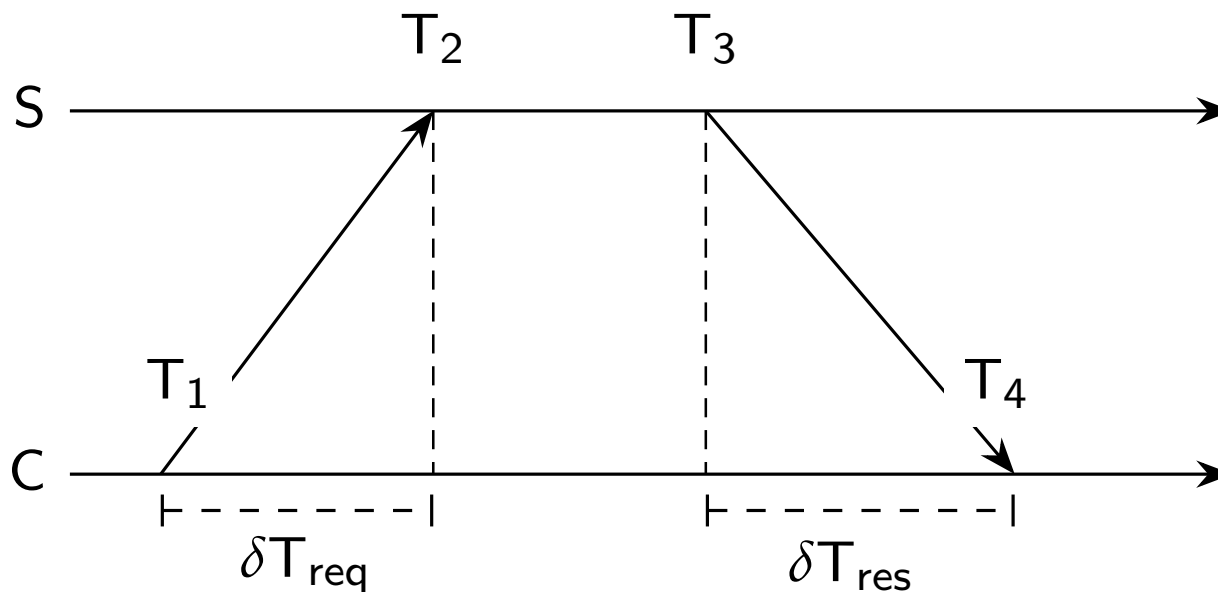
Steen and Tanenbaum (2023)

- **Oscillator Stability:** The actual frequency $F(t)$ must not deviate from the ideal frequency F by more than a factor of ρ .
- **Synchronization Frequency:** To maintain a precision π , clocks must be resynchronized (in software) at least every $\frac{\pi}{2\rho}$ seconds.



Network Time Protocol (NTP)

- **Approach:** Clients contact a time server to estimate the current time and offset.



- **Measurement Process:**

1. A sends a request to B , timestamped at T_1 .
2. B records the time of receipt T_2 .
3. B returns a response at T_3 (including T_1 and T_2).
4. A records the time of the response's arrival, T_4 .

- **Calculations:**

Estimated Offset (θ)

$$\theta = \frac{(T_2 - T_1) + (T_3 - T_4)}{2}$$

Estimated Delay (δ)

$$\delta = \frac{(T_4 - T_1) - (T_3 - T_2)}{2}$$

- **Clock Adjustment:** If A 's clock is fast ($\theta < 0$), the correction is introduced gradually by slowing down the software timer until the clocks match – **no setting clock backward!!**

NTP: Hierarchical Organization and Strata

- NTP servers are organized into layers (strata) based on their proximity to a reference clock
 - **Stratum 0**: High-precision clocks (e.g., atomic clocks or GPS receivers)
 - **Stratum 1**: Primary time servers directly connected to Stratum 0 devices
 - **Stratum k** : Servers that synchronize with Stratum $k - 1$ servers
- **Symmetry**: NTP is set up pairwise; a server A adjusts its clock only if B has a lower stratum level (more accurate reference)
- **Performance**: Designed to achieve worldwide accuracy within the range of 1–50 msec.

Reference Broadcast Synchronization (RBS)

Wireless and sensor networks where nodes are resource-constrained.

- **Mechanism:** RBS uses **receiver-to-receiver** synchronization.
 - A sender broadcasts a reference message m .
 - Receivers p and q record their local times of arrival $(T_{p,m}, T_{q,m})$.
 - Receivers exchange these timestamps to calculate their relative offset.
- **Advantage:** Eliminates nondeterminism from the sender's network access time and message construction time.
- **Drift Compensation:** Uses linear regression over multiple broadcasts to model offset as a function of time ($\text{Offset}[p, q](t) = \alpha t + \beta$).

Case Study: Google TrueTime

Implemented for the Spanner globally distributed database to support transactions.

- **Innovation:** Instead of a single timestamp, it provides a time interval $[T_{\text{lwb}}, T_{\text{upb}}]$ with a guaranteed error bound.
- **Operations:**
 - `TT.now()`: Returns the interval $[T_{\text{lwb}}, T_{\text{upb}}]$.
 - `TT.after(t)`: True if t has definitely passed.
 - `TT.before(t)`: True if t has definitely not arrived.
- **Implementation:** Uses specialized “time-master” machines equipped with GPS receivers and atomic clocks, maintaining a typical uncertainty ε of approximately 6ms.

Logical Clocks

Logical Clocks

For many distributed applications, it is not essential that clocks be accurate relative to UTC. What matters is that all processes agree on the **order** in which events occur.

- **Logical Clocks:** When the focus is on the order of events rather than physical time, the mechanisms used are referred to as logical clocks
- **Lamport's Principle:** Lamport (1978) established that synchronization does not need to be absolute:
 - **Interaction-Based:** If two processes never interact, their clocks do not need to be synchronized because the discrepancy is unobservable
 - **Event Ordering:** The critical factor is reaching a consensus on the order in which events occur, rather than the absolute timestamp of each event

Lamport's Logical Clocks

In many distributed systems, absolute physical time is less important than the **order** in which events occur.

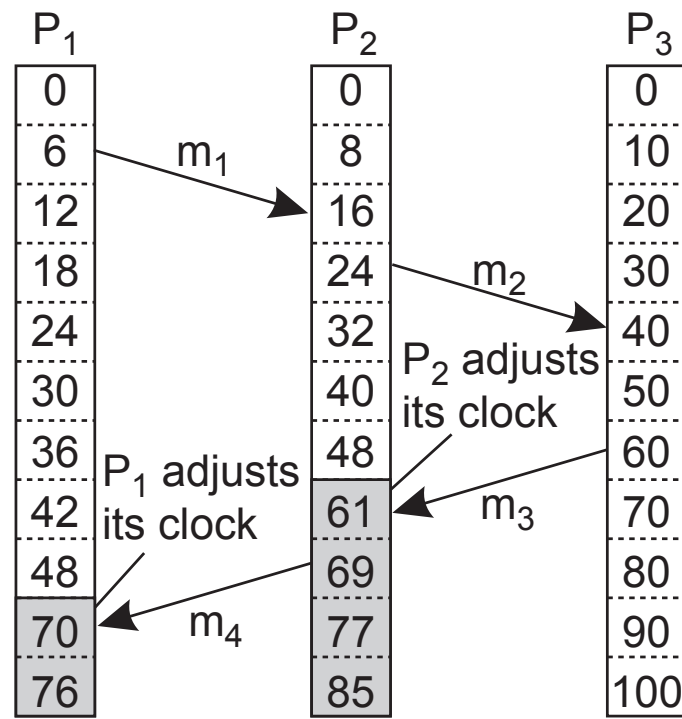
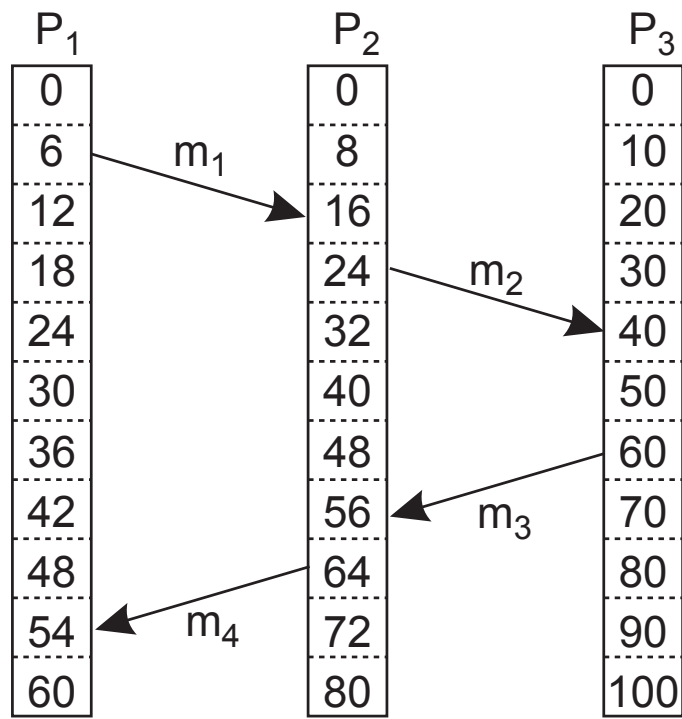
- **The Solution:** Lamport's logical clocks provide a mechanism to assign a numerical value to each event such that the “happens-before” relationship is preserved.
- **Happens-Before Relation ($\rightarrow$):**
 - If a and b occur in the same process and a comes before b , then $a \rightarrow b$.
 - If a is the sending of a message and b is its receipt, then $a \rightarrow b$.
 - If $a \rightarrow b$ and $b \rightarrow c$, then $a \rightarrow c$ (transitivity).
- **Concurrency:** If neither $a \rightarrow b$ nor $b \rightarrow a$, the events are concurrent ($a \parallel b$).

Lamport's Logical Clocks: Implementation

- **Local Clock:** Each process P_i maintains a local time value C_i .
- **Rule 1 (Internal Events):** Before executing an event (internal, send, or receive), P_i increments C_i : $C_i \leftarrow C_i + 1$
- **Rule 2 (Message Passing):**
 - When P_i sends message m , it attaches the current C_i as the timestamp $T(m)$
 - Upon receiving m with timestamp $T(m)$, process P_j updates its clock:
$$C_j \leftarrow \max\{C_j, T(m)\} + 1$$
- **Causal Property:** If $a \rightarrow b$, then $C(a) < C(b)$.
- **Limitation:** The converse is not true; $C(a) < C(b)$ does **not** imply $a \rightarrow b$.

Lamport's Logical Clocks: Implementation

Steen and Tanenbaum (2023)



Lamport's Logical Clocks: Total Ordering

- **Partial vs. Total Order:** Lamport clocks provide a partial order because concurrent events might have the same or arbitrary timestamps
- **Tie-Breaking:** To create a total order, we append the process ID to the timestamp: (C_i, i)
- **Comparison Rule:** $(a, i) < (b, j)$ if and only if:
 1. $C(a) < C(b)$, OR
 2. $C(a) = C(b)$ and $i < j$
- **Application:** Total ordering is essential for distributed mutual exclusion and ensuring all replicas apply updates in the exact same sequence

Lamport's Logical Clocks: Total Ordering Example

- **Scenario:** Multiple replicas must receive and apply updates in the same order to remain consistent
- **Mechanism:**
 1. A process multicasts a message m with a timestamp $T(m)$ to all others (including itself)
 2. Receivers put the message in a local queue ordered by timestamp and send an acknowledgment (ACK)
 3. A message is “delivered” to the application only when:
 - It is at the head of the queue
 - It has been acknowledged by every other process in the system
- **Requirement:** This algorithm assumes communication is reliable and FIFO-ordered

Vector Clocks

The Lamport Clocks have a limitation: while $a \rightarrow b$ implies $C(a) < C(b)$, the reverse is not true. Given two timestamps $C(a) < C(b)$, we cannot conclude that a causally preceded b .

- **The Purpose of Vector Clocks:** To provide a mechanism where the relation between timestamps directly reflects the causal relationship between events
- **Definition:** In a system with n processes, a vector clock VC is an array of n integers
 - Each process P_i maintains its own vector VC_i
- $VC_{i[j]}$ represents the number of events that process P_i knows have occurred at process P_j

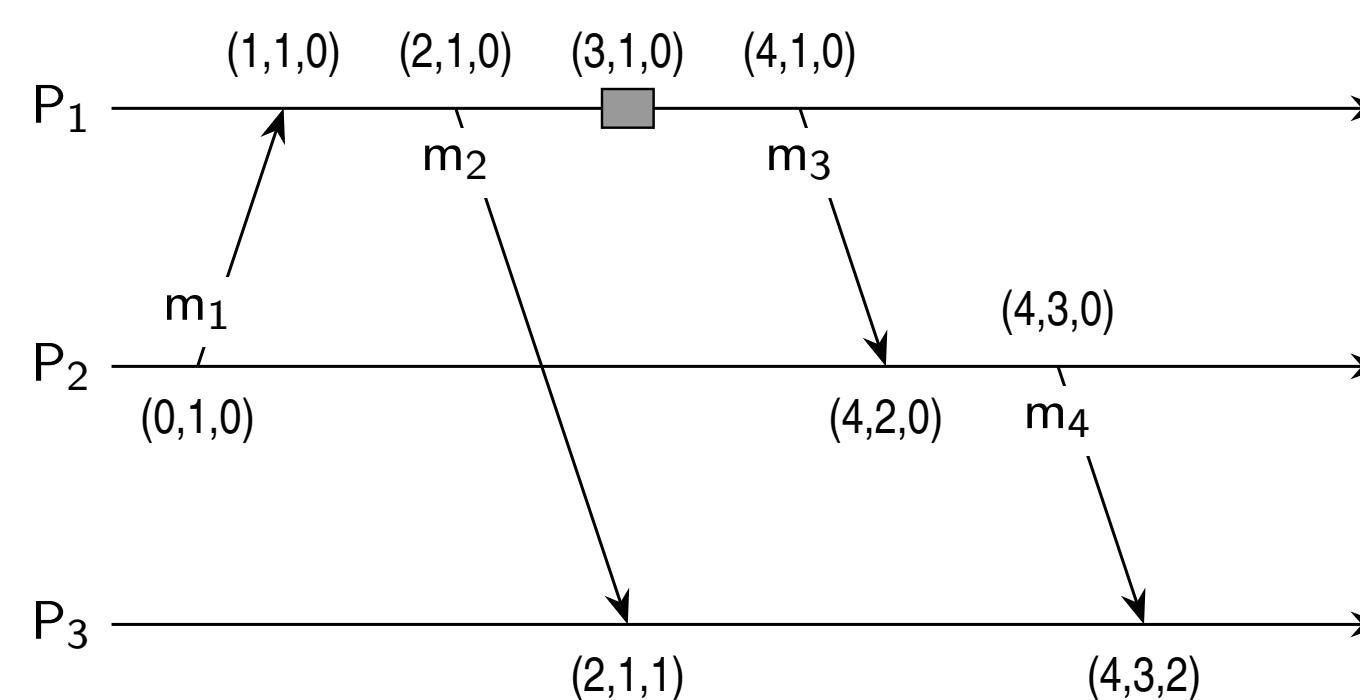
Vector Clocks: Implementation

- **Initialization:** For all i , $VC_{i[j]} = 0$ for $j = 1, \dots, n$.
- **Rule 1 (Local Event):** Before P_i executes an event, it increments its own entry:
$$VC_{i[i]} \leftarrow VC_{i[i]} + 1$$
- **Rule 2 (Message Passing):**
 - When P_i sends message m , it attaches the entire vector VC_i as a timestamp t_m
 - Upon receiving m with timestamp t_m , process P_j updates its vector:
$$VC_{j[k]} \leftarrow \max\{VC_{j[k]}, t_{m[k]}\} \text{ for all } k$$
 - P_j then increments $VC_{j[j]}$ and delivers the message

Vector Clocks: Comparing Timestamps

- **Equality:** $VC(a) = VC(b)$ iff $VC(a)[i] = VC(b)[i]$ for all i .
- **Less Than or Equal:** $VC(a) \leq VC(b)$ iff $VC(a)[i] \leq VC(b)[i]$ for all i .
- **Causal Ordering:** a causally precedes b ($a \rightarrow b$) if and only if:
 1. $VC(a) \leq VC(b)$, AND
 2. $VC(a) \neq VC(b)$ (there exists at least one j such that $VC(a)[j] < VC(b)[j]$)
- **Concurrency:** Events a and b are concurrent ($a \parallel b$) if:
 - neither $VC(a) \leq VC(b)$ nor $VC(b) \leq VC(a)$

Vector Clocks: Comparing Timestamps



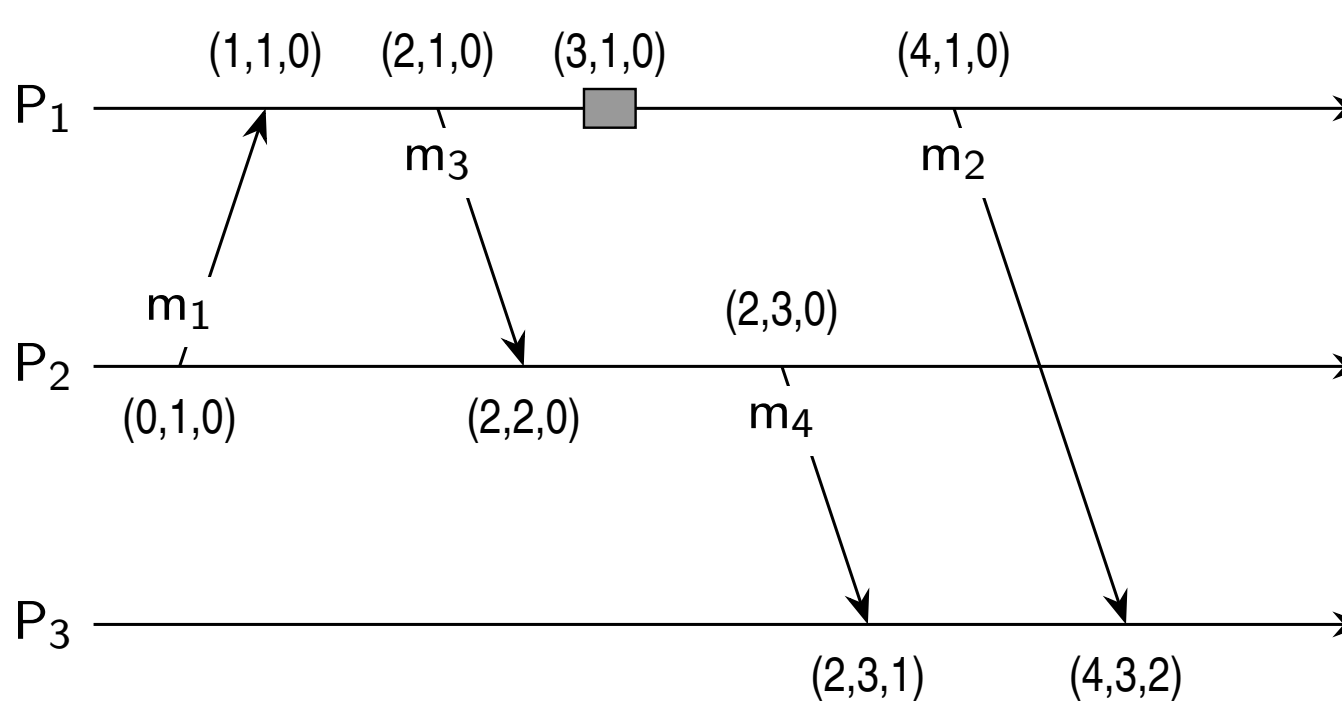
$ts(m_2) < ts(m_4)$ is True

$ts(m_4) < ts(m_2)$ is False

Conclusion: m_2 may causally precede m_4

Steen and Tanenbaum (2023)

Vector Clocks: Comparing Timestamps



$ts(m_2) < ts(m_4)$ is False

$ts(m_4) < ts(m_2)$ is False

Conclusion: m_2 and m_4 may conflict

Steen and Tanenbaum (2023)

Mutual Exclusion

Mutual Exclusion

Mutual exclusion ensures that when one process is accessing a shared resource (the **critical section**), no other process is allowed to do the same.

- **The Distributed Challenge:**

- In a single-OS system, mutual exclusion is often managed via semaphores or monitors
- In a distributed system, the lack of shared memory and a global clock requires processes to coordinate via message passing

Mutual Exclusion: Requirements & Categories

- **Requirements:**

1. **Safety:** At most one process can be in the critical section at any time
2. **Liveness:** Every request to enter/exit the critical section eventually succeeds (no deadlock or starvation).
3. **Ordering:** If req. a happened-before req. b , then a should be granted access before b

- **Algorithm Categories:**

- **Permission-based:** A process wanting to enter a critical section must get permission from other processes.
- **Token-based:** A special message (a token) is passed around; only the holder can enter the critical section.

Mutual Exclusion: A Centralized Algorithm

The most straightforward way to achieve mutual exclusion in a distributed system is to simulate a single-processor system by using a coordinator.

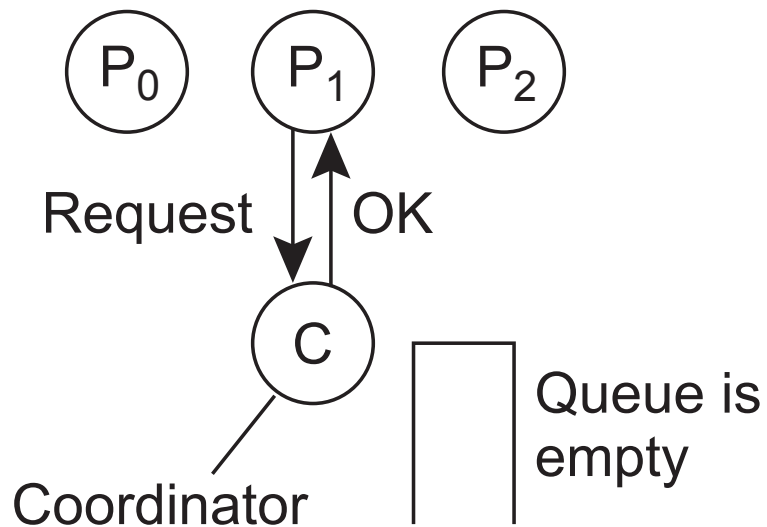
- **Roles:**
 - **Coordinator:** Manages access to the resource
 - **Participants:** Processes that wish to enter the critical section
- **Primary Mechanism:** Processes must request and receive explicit permission from the coordinator before entering their critical section

A Centralized Algorithm: Protocol Steps

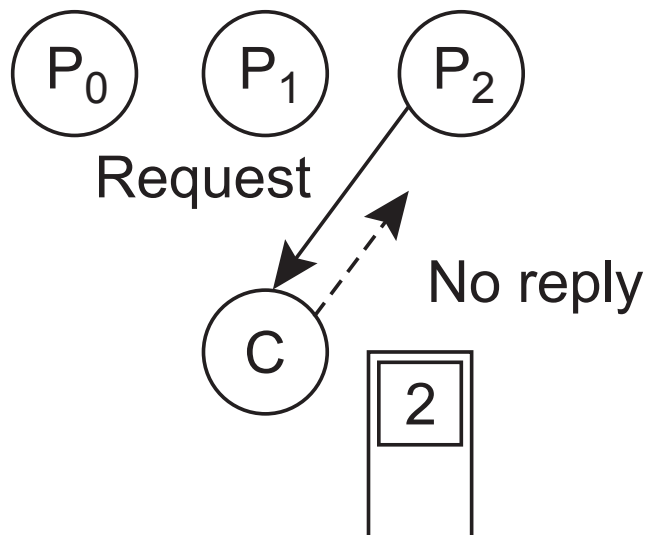
1. **Requesting Entry:** A process P_i sends a **REQUEST** message to the coordinator
 - specifying which resource it wants to access
2. **Granting Permission:**
 - **Available:** The coordinator sends a **GRANT** message immediately
 - **Occupied:** The coordinator queues P_i 's request without replying
3. **Exiting:** When P_i finishes, it sends a **RELEASE** message to the coordinator.
4. **Managing the Queue:** The coordinator takes the first request from its queue and sends a **GRANT** message to that process.

A Centralized Algorithm: Protocol Steps

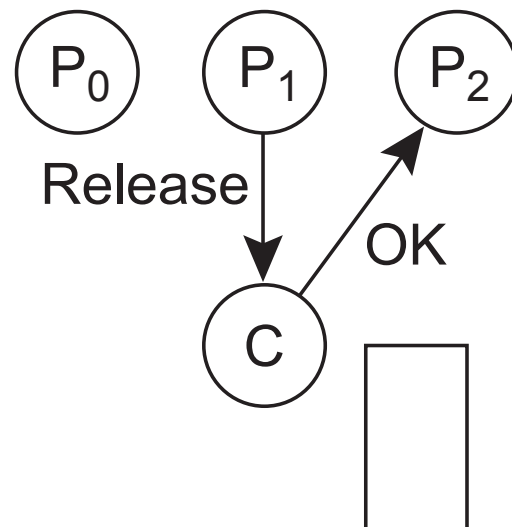
Steen and Tanenbaum (2023)



(a)



(b)



(c)

A Centralized Algorithm: Advantages and Properties

- **Fairness:** By using a FIFO queue, the coordinator ensures that no process is starved
 - everyone eventually gets a turn in the order they requested it
- **Efficiency:** Only three messages are required per entry into a critical section:
 - **REQUEST**
 - **GRANT**
 - **RELEASE**
- **Simplicity:** The logic is easy to implement and verify, as the state is maintained in one location

A Centralized Algorithm: Shortcomings and Vulnerabilities

- **Single Point of Failure:** If the coordinator crashes, the entire system can hang
 - Processes waiting for a **GRANT** will wait forever, unable to distinguish between a “denied” request (because the resource is busy) and a “lost” request (because the coordinator is dead)
- **Performance Bottleneck:** In a very large system with frequent requests, the single coordinator can become a bottleneck for the entire network.
- **Blocking vs. Non-blocking:** The standard algorithm is blocking; if a process does not receive a reply, it remains suspended.

Mutual Exclusion: A Distributed Algorithm

The **Ricart-Agrawala algorithm** aims to provide mutual exclusion without a central coordinator. It relies on total ordering of events using Lamport's logical clocks.

- **Requirements:**

- All processes have unique IDs
- All processes maintain a Lamport logical clock
- Communication is reliable (no lost messages)

- **General Principle:** A process can only enter a critical section if it receives a “permission” message from **every other** process in the group.

A Distributed Algorithm: Requesting Access

- **Initiation:** When process P_i wants to enter a critical section:
 1. It builds a message containing the name of the resource, its process ID, and its current logical time T
 2. It multicasts this message to all other processes in the system
- **Wait State:** P_i then waits until every other process has sent an “OK” message back

A Distributed Algorithm: Responding to a Request

When process P_j receives a request from P_i with timestamp T :

- **Case 1 (Idle):** P_j is not interested and sends **OK** to P_i immediately
- **Case 2 (Busy):** P_j is in the critical section; it queues the P_i request without replying
- **Case 3 (Conflict):** P_j also wants entry; it compares the message timestamp T with its own
 - The process with the **lower** timestamp wins.
 - If P_i has a lower timestamp, P_j sends an **OK** message
 - If P_j has a lower timestamp, it queues P_i 's request and sends nothing

A Distributed Algorithm: Exiting the Critical Section

- **Action:** When a process P_i finishes its task in the critical section:
 1. It looks at its internal queue of pending requests
 2. It sends an **OK** message to every process currently in that queue
 3. It clears the queue
- **Outcome:** This unblocks the next process in the total ordering of requests, allowing it to collect its final required **OK** and enter the resource

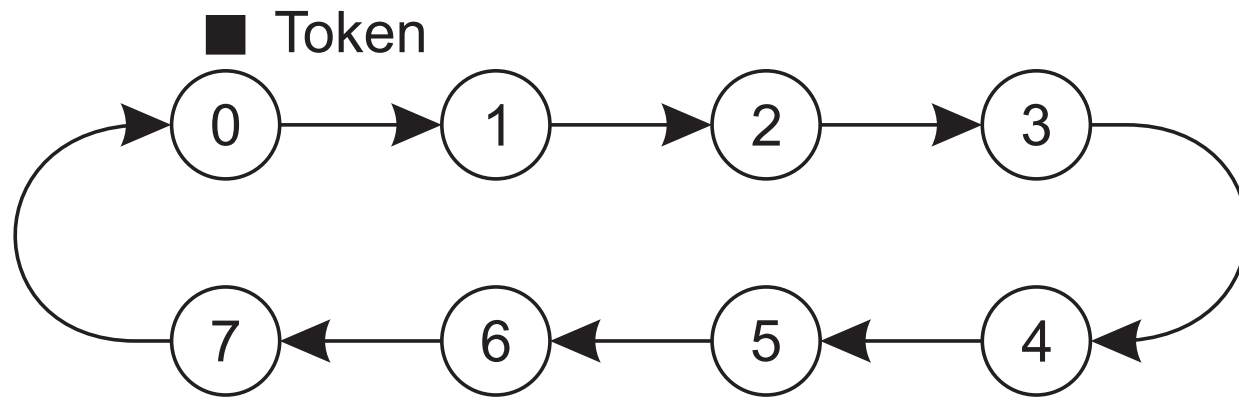
A Distributed Algorithm: Performance and Fault Tolerance

- **Message Complexity:** For n processes, it requires $n - 1$ requests and $n - 1$ permissions, totaling $2(n - 1)$ messages.
- **Critical Weakness:** This algorithm is “ n points of failure”
 - If any single process crashes, it will fail to respond to requests
 - Other processes will interpret this silence as “permission denied” or simply wait forever, effectively blocking the entire system
- **Detection:** To function correctly, processes must distinguish between a busy process and a dead process, typically requiring a timeout or a reliable failure detector

Mutual Exclusion: A Token Ring Algorithm

Processes are organized into a logical ring, regardless of their physical network topology.

- **Mechanism:** A single, special message called a **token** is circulated around the ring
- **Access Rule:** A process is only permitted to enter its critical section if it currently holds the token



Steen and Tanenbaum (2023)

A Token Ring Algorithm: Protocol Steps

1. **Initialization:** When the system starts, process 0 is given the token
2. **Token Passing:** The token is passed from process P_i to $P_{(i+1) \bmod n}$
3. **Resource Usage:**
 - Upon receiving the token, P_i checks if it needs to enter the critical section
 - **If Yes:** It enters the critical section, performs its work, and exits
 - **If No:** It immediately passes the token to its neighbor
4. **Post-Exit:** After exiting the critical section, the process must pass the token
 - It cannot immediately re-enter the critical section using the same token

A Token Ring Algorithm: Advantages and Performance

- **Safety:** Since there is only one token, only one process can be in the critical section at any time, satisfying the safety requirement
- **Liveness:** In a ring of n processes, a process will wait at most $n - 1$ other processes before the token returns to it
 - Starvation is impossible
- **Message Overhead:**
 - If no process wants the critical section, the token circulates continuously, consuming bandwidth
 - When a process wants to enter, the delay ranges from 0 (token just arrived) to $n - 1$ message transfers

A Token Ring Algorithm: Reliability and Fault Tolerance

- **Lost Token:** If the token is lost (e.g., due to a process crashing while holding it), it must be regenerated
 - Detecting a lost token is difficult, as one cannot easily distinguish between a “slow” token and a “lost” one
- **Process Failure:** If a process in the ring crashes, the ring is broken
 - To recover, the system must detect the dead process and logically bypass it by connecting its predecessor to its successor
 - This requires every process to maintain knowledge of the current ring membership.

Mutual Exclusion: A Decentralized Algorithm

Permission-based and token-based algorithms often suffer from being “n points of failure.” The decentralized approach aims for a more robust, probabilistic solution based on voting.

- **System Assumptions:**
 - The resource is replicated n times
 - Each replica i has its own coordinator C_i
 - Access is granted via a voting mechanism
- **The Rule:** To access the resource, a process must obtain a majority of $m > \frac{n}{2}$ votes from the n coordinators

A Decentralized Algorithm: Protocol Steps

- **Requesting Access:**

1. A process P sends a request to all n coordinators
2. Each coordinator C_i grants permission only if it has not already granted it to another process
3. P enters the critical section only if it receives at least m **OK** responses

- **Handling Denials:**

- If a coordinator has already granted its vote, it responds with a **DENY** message
- If a process fails to get m votes, it must release all currently held votes and back off for a random interval before retrying

- **Releasing Access:** Upon exiting, the process sends a **RELEASE** to all coordinators.

A Decentralized Algorithm: Reliability

- **Resilience:** Systems can tolerate some coordinator failures because only a majority is required for a lock
- **State Loss:** Upon crashing, a coordinator “forgets” its current state and reboots
- **Violation Risk:** Simultaneous crashes/reboots create a small but non-zero risk of two processes claiming a majority
- **Reboot Conflict:** A rebooted coordinator might grant a vote to a new process while the old lock holder is still active
- **Probability:** The risk of simultaneous resource access depends on the crash rate (f) and the critical section duration

A Decentralized Algorithm: Pros & Cons

- **Pros:**
 - Highly decentralized; no single coordinator is the master
 - Better fault tolerance than the centralized or Ricart-Agrawala models
- **Cons:**
 - **Complexity** of managing multiple replicas
 - **Wasteful:** if a process fails to get a majority, it must release everything and start over (Livelock potential)
 - **Safety** is no longer absolute (100% guaranteed) but rather extremely likely

Leader Election

Election Algorithms

Many distributed algorithms require a specific process to act as a coordinator, initiator, or leader. If the current coordinator fails, the system must elect a new one.

- **Basic Assumptions:**
 - Each process P_i has a unique identifier (ID)
 - Every process knows the IDs of all other processes in the system
 - Processes do not know which ones are currently “up” or “down”
- **General Objective:** To ensure that when an election concludes, all non-faulty processes agree on exactly one unique coordinator
- **Criteria for Selection:** Typically, the process with the highest ID is elected as the new coordinator

The Bully Algorithm

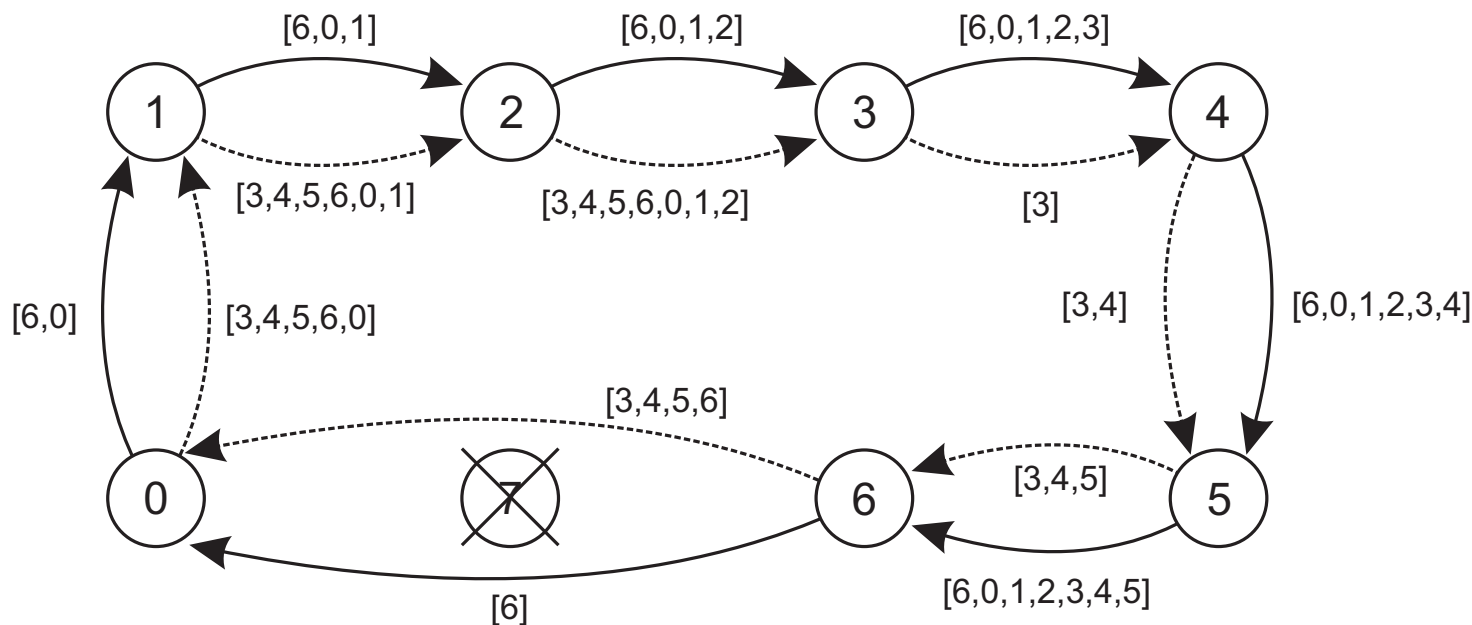
- **Concept:** When a process notices the coordinator is no longer responding, it initiates an election to “bully” processes with lower IDs into submission
- **The Protocol:**
 1. P_i sends an **ELECTION** message to all processes with higher IDs (P_j where $j > i$)
 2. If no one responds, P_i wins and sends a **COORDINATOR** message to everyone
 3. If a higher-ID process P_j responds with an **OK**, P_j takes over the election, and P_i 's job is done
- **Recovery:** If a process that was previously down (and has a high ID) recovers, it immediately holds an election
 - It will “bully” the current coordinator and take over

A Ring Algorithm

- **Concept:** Based on a logical ring structure, but unlike the token ring, this does not use a token, it uses message passing to find the highest ID
- **The Protocol:**
 1. A process P_i detects failure and sends an **ELECTION** msg (with its ID) to its successor
 2. If a successor is unresponsive, the sender skips to the next active process in the ring
 3. Each receiver adds its own ID to the list in the message and passes it on
- **Termination:**
 1. When the message returns to P_i , it sees its own ID in the list (full circle)
 2. Message type changes to **COORDINATOR**, P with the highest ID is the new leader
 3. This **COORDINATOR** message circulates once more to inform everyone

A Ring Algorithm: Example

P_7 crashes – P_6 and P_3 starts election



Steen and Tanenbaum (2023)

Elections in Large-Scale Systems

- **The Challenge:** Algorithms like Bully or Ring require $O(n)$ or $O(n^2)$ messages and knowledge of all process IDs, which is not scalable for massive networks.
- **Hierarchical Approaches:**
 - Nodes are divided into groups
 - e.g., clusters or DHT-based structures
 - A local leader is elected within each group
 - These local leaders then elect a global leader among themselves
- **Super-peers:** In P2P systems, some nodes (super-peers) are selected based on high bandwidth, availability, and processing power to handle coordination tasks for smaller peers

Elections in Wireless Environments

- **Unique Constraints:** Limited battery, fluctuating connectivity, and unpredictable topology make standard election algorithms unreliable
- **Mechanism (Vasudevan et al.):**
 1. A source node initiates an election by sending an **ELECTION** message to neighbors
 2. This builds a spanning tree: the first node to send the message becomes the “parent”
 3. Neighbors forward the message to their own neighbors (except their parent)
 4. Once a node has heard from all neighbors, it reports its “best” candidate (highest ID) back to its parent
- **Result:** The source eventually receives reports from all reachable nodes and multicasts the identity of the winner

Gossip-based Coordination

Aggregation

Aggregation is a fundamental coordination task in large-scale distributed systems where the goal is to compute a collective value (e.g., average, sum, maximum, or minimum) from data held by individual nodes.

- **Decentralized Approach:** Rather than sending all data to a central server, nodes use gossip-based (epidemic) protocols to compute the result through local interactions
- **Mechanism:**
 - Each node P_i starts with a local value v_i
 - Periodically, it selects a random neighbor P_j and both exchange and update their values based on a specific function

Aggregation: Computing a Distributed Average

- **Protocol:**
 1. Node P_i initiates an exchange with a random neighbor P_j
 2. Both nodes update their local values: $v_i, v_j \leftarrow \frac{v_i + v_j}{2}$
- **Convergence:** As this process repeats across the network, the local values of all nodes eventually converge to the true global average ($\frac{1}{n} \sum v_i$)
- **Speed:** In a connected graph, the variance among values decreases exponentially, leading to very rapid convergence even in massive systems

Aggregation: Computing other Aggregate Values

- **Maximum/Minimum:**
 - **Protocol:** $v_i, v_j \leftarrow \max(v_i, v_j)$
 - **Result:** After enough rounds, every node will hold the max value present in the system
- **Sum Estimation:**
 - One node starts with value 1, all others start with 0
 - Nodes run the “Average” protocol
 - Eventually, all nodes hold $\frac{1}{n}$
- **Network Size (n):** By running the average protocol where one node starts with 1 and others with 0, each node ends with $\frac{1}{n}$, allowing them to estimate the total number of nodes in the system.

Aggregation: Robustness and Properties

- **Fault Tolerance:** Gossip protocols are highly resilient to node failures
 - If a node crashes, the remaining nodes continue to exchange values, and the aggregate value (like a maximum) remains unaffected
- **Fixed Values:** If the protocol is used to compute an average and nodes fail, the final average might be slightly skewed, but the system does not crash or stop
- **Scalability:** The communication load per node is constant regardless of the total system size, making it ideal for extremely large, dynamic networks

A Peer-Sampling Service

Many gossip-based coordination algorithms (like aggregation) assume that a node can select a neighbor **uniformly at random** from the entire set of nodes in the system.

- **The Scalability Problem:** In a massive distributed system, it is impractical for every node to maintain a complete, up-to-date list of all other nodes
- **The Solution:** A peer-sampling service provides each node with a small, frequently updated subset of the system (a **partial view**) that approximates a random sample of the entire population.

A Peer-Sampling Service: Gossiping

Nodes use an epidemic (gossip) protocol to exchange and refresh their partial views.

- **View Exchange Protocol:**

1. Periodically, node P selects a neighbor Q from its current partial view
2. P and Q exchange parts of their respective partial views
3. Both nodes update their views by merging the received information and discarding older or redundant entries

- **Result:** This continuous shuffling ensures that the partial views do not become static and that they eventually contain a representative sample of the network

A Peer-Sampling Service: Managing the Partial View

- **Entry Structure:** Each entry in a partial view typically consists of a node's address and an “age” (timestamp or hop-count)
- **Update Policy:**
 - **Selection:** Which nodes to send to a neighbor?
 - e.g., the youngest/freshest entries
 - **Retention:** Which nodes to keep after a merge?
 - e.g., discarding duplicates and the oldest entries to ensure “dead” nodes are eventually removed
- **Self-Healing:** By prioritizing “fresh” entries, the service naturally purges references to nodes that have crashed, as their entries' ages will keep increasing without being refreshed

A Peer-Sampling Service: Properties and Robustness

- **Graph Connectivity:** A critical goal of the peer-sampling service is to ensure the logical graph formed by partial views remains a “small-world” network and, most importantly, **does not become partitioned**
- **Resilience:** The service is highly robust against massive node failures
 - Even if a large percentage of nodes drop out, the remaining nodes will eventually exchange enough information to “clean” their views of dead links
- **Independence:** The peer-sampling service acts as a fundamental “building block” or middleware
 - Other coordination tasks (like aggregation or overlay construction) sit on top of it, requesting a random peer whenever needed

Distributed Event Matching

Distributed Event Matching

Event matching (or notification filtering) is the core process in **publish-subscribe** systems that determines which subscribers should receive a specific notification.

- **The Matching Process:**

1. **Subscription (S):** A process specifies the events it is interested in
 2. **Notification (N):** A process publishes a notification of an event
 3. **Matching Function:** The system executes $\text{match}(S, N)$ to find pairs where the condition is true
- **Scalability Challenges:** Distributing the matching load across multiple servers without full global knowledge

Distributed Event Matching: Centralized and Divided

- **Centralized Server:** All subscriptions and notifications are sent to a single server
 - While simple it is a single point of failure and a bottleneck
- **Dividing Work via Rendezvous Nodes:** To scale, the system uses mapping functions to assign work to a subset of servers:
 - $\text{sub2node}(S)$: Maps a subscription to a set of rendezvous nodes.
 - $\text{not2node}(N)$: Maps a notification to a set of rendezvous nodes.
 - **Constraint:** For any S and N that match, their rendezvous node sets must intersect:
 - $\text{sub2node}(S) \cap \text{not2node}(N) \neq \emptyset$.
- **Distributed Brokers:** Brokers organized in an overlay network can use flooding:
 - **Option A:** Store every subscription at every broker; notifications are sent to just one
 - **Option B:** Store subscriptions at one broker; broadcast notifications to all

Distributed Event Matching: TIB/Rendezvous Example

- **Subject-Based Filtering:** Notifications are tagged with compound keywords
 - e.g., news.comp.os.books
- **Subscribing:** Users use wildcards (e.g., news.comp.*.books) to indicate interest
- **Implementation:**
 - Each host runs a **rendezvous daemon**
 - When a message is published, it is multicasted to all hosts on the local network
 - Local daemons check the subject against local subscriptions and deliver matches to app
- **Efficiency:** Uses hardware broadcasting or IP-multicasting when available to reduce network load

Location Systems

Location Systems: Coordination and Proximity

In large-scale distributed systems, proximity matters. Overlay neighbors may be logically close but physically distant, leading to inefficient communication.

- **Role in Coordination:** Location-based techniques help coordinate the physical placement of processes and optimize their communication paths
- **Types of Systems:**
 - **Absolute Positioning:** Determining exact geographical coordinates (e.g., GPS)
 - **Relative/Indoor Positioning:** Using local signals (e.g., WiFi) when GPS is unavailable
 - **Logical Positioning:** Assigning coordinates in a geometric space to reflect network performance (e.g., latency)

Global Positioning System (GPS)

- **Infrastructure:** A satellite-based system using up to 72 satellites orbiting at $\approx 20,000$ km
- **Mechanism:** Each satellite contains highly accurate atomic clocks and continuously broadcasts its position and the current timestamp
- **Principles of Positioning:**
 - **Two-Dimensional:** Requires distance to 3 satellites (intersection of circles)
 - **Three-Dimensional:** Requires distance to 4 satellites to determine longitude, latitude, altitude, and a time offset (Δr)
- **Calculations:** Distance is estimated as $\tilde{d}_i = d_i + c \cdot \Delta r$, where c is the speed of light and Δr is the receiver's clock deviation.
- **Error Sources:** Finite clock accuracy, atmospheric fluctuations affecting signal speed, and imprecise satellite orbital data.

Positioning When GPS is Not an Option

GPS signals generally cannot penetrate buildings, requiring alternative indoor solutions.

- **WiFi-Based Positioning:** Uses the location of known WiFi access points (APs) to estimate the receiver's position via trilateration
- **War Driving:** The process of populating databases by driving or walking through areas with a WiFi-enabled device and a GPS receiver to record AP coordinates
- **Centroid Calculation:** A simple method to estimate an AP's location by averaging all GPS detection points where it was observed: $\vec{x}_{AP} = \frac{\sum_i^N \vec{x}_i}{N}$
- **Accuracy Factors:** GPS accuracy at detection points, non-uniform transmission ranges, and the high rate of AP turnover

Logical Positioning of Nodes

- **Network Coordinates System (NCS):** Assigns nodes coordinates in an m -dimensional space where the geometric distance $\hat{d}(P, Q)$ reflects a network metric, typically latency.
- **Applications:**
 - **Client Redirection:** CDNs can redirect clients to the “closest” server based on latency
 - **Optimal Placement:** Finding the best positions for new replicas to minimize average response time
 - **Position-Based Routing:** Forwarding messages to neighbors that are geometrically closer to the destination
- **Triangle Inequality Constraint:** Real-world Internet latencies often violate $d(P, R) \leq d(P, Q) + d(Q, R)$, leading to inherent inconsistencies in coordinates

Centralized and Decentralized NCS Algorithms

- **Centralized (Landmarks):**
 - Uses N special nodes (landmarks) that measure pairwise latencies
 - A central node minimizes an aggregated error function to assign landmarks's coordinates
 - Other nodes measure their distance to landmarks to compute their own positions
- **Decentralized (Vivaldi):**
 - Models the system as physical springs between nodes
 - The difference $|\tilde{d}(P, Q) - \hat{d}(P, Q)|$ indicates the displacement from a “rest” state
 - **Protocol:** Nodes measure latency to a peer, compute a “force” (push or pull), and adjust their own coordinates to minimize the local error
 - **Adaptive Step (δ):** Ensures convergence by adjusting the movement size based on the current error magnitude

Resources

Resources

- M. van Steen and A.S. Tanenbaum, Distributed Systems, 4th ed., distributed-systems.net, 2023.